

Contents

Exercise 3 — Fit, regularise, and say what you believe	1
Goal	1
Dataset	1
Tasks	2
1. Explore and prepare (build on Exercise 2)	2
2. Baseline and first model	2
3. Does a straight line fit?	2
4. Choose a penalty — without using the test set	2
5. Ridge against Lasso	2
6. Which coefficients do you believe?	3
7. Conclude	3
What to hand in	3
Assessment criteria	3
Automatic deductions	4
Hints	4

Exercise 3 — Fit, regularise, and say what you believe

Set: Friday 9 October 2026 · **Due:** Friday 16 October 2026, start of Lesson 4

Roughly 2–3 hours. Work alone; discussing ideas is fine, sharing notebooks is not.

Goal

Fit a regression model to a dataset you have not seen, tune a penalty without peeking at your test set, and — the part that carries the most marks — **say which coefficients you would be willing to defend, and why.**

Dataset

Bike sharing demand: **hourly** hire counts against weather and calendar features — 17,379 rows and 12 columns, two years of a city bike scheme. Note the **hour** column, and expect it to matter.

```
from sklearn.datasets import fetch_openml
```

```
bikes = fetch_openml(name="Bike_Sharing_Demand", version=2, as_frame=True)
X, y = bikes.data, bikes.target
```

The first call downloads and caches it; afterwards it works offline. If your network blocks it, the course image has it cached.

Two things to notice before you begin:

- **Several features are strongly related to each other** — temperature and “feels like” temperature, season and month. Section 7 of the handout is about exactly this situation.

- **The target is a count**, not an unbounded quantity. A linear model can predict negative demand, which is impossible. You are not required to fix that, but you are required to notice it, count how many of your predictions are negative, and say what you would do.
 - **The penalty may buy you very little here, and that is a real answer.** With thirteen thousand training rows and twenty columns there is not much overfitting to remove, so do not tune until the number moves. Report the sweep you ran and what it showed. Task 4 is marked on the *procedure* — choosing without touching the test set — not on finding an improvement.
-

Tasks

1. Explore and prepare (build on Exercise 2)

Load, inspect, split, and build a `ColumnTransformer` handling the numeric and categorical columns. This is Lesson 2's work; do it quickly.

State your split proportion and why.

2. Baseline and first model

Report the baseline — predicting the mean — as root mean squared error (RMSE), the square root of the mean squared error.

Then fit an unregularised `LinearRegression` inside your pipeline and report train and test RMSE.

Answer in a sentence: is the gap between them large, and what would a large gap mean?

3. Does a straight line fit?

Plot the residuals against the two most important numeric features.

If you see structure, say what it is and add polynomial features for that column only. Report whether it helped.

4. Choose a penalty — without using the test set

Fit `Ridge` for at least six values of the penalty — the `alpha=` argument in scikit-learn — spanning several orders of magnitude.

You may **not** choose by looking at test performance. Instead, hold out a **validation** set from your training data, choose on that, and only then report the test score for your chosen model.

Answer in a paragraph: why is choosing on the test set a problem, given that you would only look at it once? This is Lesson 1's argument applied, and Lesson 5 will give you better machinery — the point today is to feel why it is needed.

5. Ridge against Lasso

Fit `Lasso` at a `penalty` that removes at least two features.

Report which features survived and in what order they were dropped as `penalty` rose.

Answer in a sentence: does the surviving set match what you would have guessed from the data, and what would change if you had scaled differently?

6. Which coefficients do you believe?

Produce the correlation matrix of your numeric features.

Then, for **three** coefficients of your final model, state whether you would defend that number to a colleague, and why or why not. At least one should be a coefficient you do **not** trust.

This is the task that carries the most marks. A correct model with a naive reading of its coefficients scores lower than a modest model read carefully.

7. Conclude

In no more than 150 words: what does your model predict well, what does it predict badly, and what would you do with another week?

What to hand in

A single notebook named {surname}_03.ipynb that:

- runs top to bottom in the course container with no edits
 - has `random_state` fixed everywhere
 - keeps its outputs, so it can be read without being run
 - alternates markdown and code — the prose is where the reasoning lives
-

Assessment criteria

Criterion	Weight	What earns marks
Methodological correctness	40%	chosen without touching the test set; preprocessing inside the pipeline; scaling before regularisation; no leakage
Implementation	25%	Runs clean and reproducible; residual plots that support the argument
Interpretation	35%	Task 6 done seriously: coefficients defended or doubted with reasons, not restated

No marks for accuracy. A model with a worse RMSE and an honest account of which coefficients mean anything beats a better one presented uncritically.

Automatic deductions

Problem	Effect
chosen by test-set performance	Methodology capped at 40%
Regularisation applied to unscaled features	Methodology capped at 50%
Notebook does not run in the course image	Returned for resubmission

Hints

- `np.logspace(-3, 4, 8)` gives a reasonable sweep of `alpha` to pass as `alpha=`.
- For task 4, a second `train_test_split` on your training data is enough. You are hand-rolling what `GridSearchCV` will do for you in Lesson 5.
- For task 6, the handout's Section 7.2 table is the model to imitate: put the estimated coefficient next to the feature's correlation with its neighbours.
- If a coefficient has the wrong **sign** compared with what you expect, that is usually collinearity rather than a discovery. Say so.